

Překladač latinské Javy do instrukcí

Jan Vašátko, Jiří Třesohlavý

Fakulta aplikovaných věd

October 20, 2024

Příklad Javy v latině

■ Přepsání klíčových slov z angličtiny do latiny

```
1  classis ClassisSalve {
2      publicus staticus vacuum principalis(String[] argumenta) {
3          System.out.println("Salve, mundi ");
4
5          int numerus = 10;
6
7          si (numerus > 5) {
8              System.out.println("Numerus est maior quam quinque.")
9          } aliter {
10             System.out.println("Numerus non est maior quam quinque.")
11         }
12
13         pro (int i = 0; i < numerus; i++) {
14             System.out.println("Iteratio: " + i);
15         }
16     }
17 }
```

Práce s celými čísly

- Budeme volit znaménkový `int` (32-bitový)
- Zvažujeme i práci s jinými typy (`long`, `short`,
- Číselné literály mohou být de novány v násl. sou
 - Desítková – 42
 - Šestnáctková – `0xFF00FF`
 - Dvojková – `0B101`
 - Osmičková – `0377`
- Nevyhýbáme se ani římským číslicím

Práce s reál. čísl

- Pro reálná čísla využijeme typ `float` – 32-bitový single-precision formát IEEE 754
- Literály pro reálná čísla se zapisují:
 - S desetinou tečkou – `3.14159`
 - S exponentem – `3.14e6`
- Pokud je reál. číslo ve výrazu s celým číslem:
 - Celé číslo se implicitně přetypuje na reál. číslo
 - Následně se provádí aritmetika s reál. číslem

Reprezentace logic. typů

- Pro logické hodnoty použijeme datový typ `boolean`
- Tento datový typ může nabývat jen hodnot `true` a `false`

Reprezentace znaků

- Pro znakové typy použijeme dat. typ `char` – 16-bit
- Literály se zapisují jako
 - Jednotlivé znaky v uvozovkách (`'a'`)
 - Unicode sekvence
- Ve výrazech s typem `int` bude implicitně přetypováno

Reprezentace pol'

- Pole budeme vytvářet stejně jako v Javě:
 - `typ[] p = new typ[velikost];` pro vytvoření pole velikosti.
 - `typ[] p = { vyčet prvků };` pro inicializaci pole.
- Pole budou implementována pomocí referenční polí, tedy stejně jako v Javě.
- Přiřazování mezi poli probíhá přiřazením referencí, nikoli kopírováním obsahu.
- Implementujeme operátory:
 - `[]` pro přístup k prvkům pole.
 - `.length` pro zjištění délky pole.

Reprezentace řetězců

- Řetězce můžeme implementovat jako `Filum r = "retezec";`.
- V paměti budou uloženy jako `null-terminated`.
- K dispozici budou následující operátory:
 - `[]` pro přístup k jednotlivým znakům,
 - `.longitudo` pro zjištění délky řetězce,
 - `==` pro porovnání řetězců.

Operátory a řídící struktury

■ Aritmetické operátory:

- +, - (včetně unárního), *, /, %
- Inkrementace a dekrementace: ++, --
- Zkrácené přiřazení: +=, -=, *=, /=, %=

■ Relační operátory:

- <, <=, ==, >, >=

■ Logické operátory:

- &, &&, |, ||

■ Řídící struktury:

- if, if-else, switch, ternární operátor ? :
- Smyčky: for, while

Gramatika našeho jazyka

- Pravidla pro gramatiku našeho jazyka budeme brát v úvahu při tvorbě lexeru a parseru
- Téměř totožná s pravidly klasické Javy

Deklarace identifikátoru a výrazu:

```
1 Identifier
2   : [a-zA-Z_] [a-zA-Z_0-9]*
3   ;
```

```
1 expression
2   : primaryExpression // napr.: promena  cislo
3   | expression operator expression
4   | '(' expression ')
5   ;
```

Pravidla pro cyklus `for` a `if` statement

```
1  forStatement
2      : 'pro' '(' forControl ')' statement
3      ;
4
5  forControl
6      : forInit? ';' expression? ';' forUpdate?
7      ;
8
9  forInit
10     : variableDeclaration
11     | expressionList
12     ;
13
14 forUpdate
15     : expressionList
16     ;
```

```
1  ifStatement
2      : 'num' '(' expression ')' statement ('else' statement)?
3      ;
```

Deklarace metod

```
1  methodDeclaration
2      : type Identifier '(' parameterList? ')' block
3      ;
4
5  parameterList
6      : parameter (',' parameter)*
7      ;
8
9  parameter
10     : type Identifier
11     ;
12
13  block
14     : '{' blockStatement* '}'
15     ;
16
17  blockStatement
18     : variableDeclaration
19     | statement
20     ;
21
22  statement
23     : ifStatement
24     | forStatement
25     | expressionStatement
26     | returnStatement
27     ;
```

Zdroje (APA citation)

- Lexer: Antlr. (n.d.). grammars-v4/java/java9/Java9Lexer.g4 at master · antlr/grammars-v4. GitHub.
<https://github.com/antlr/grammars-v4/blob/master/java/java9/Java9Lexer.g4>
- Parser: Antlr. (n.d.). grammars-v4/java/java9/Java9Parser.g4 at master · antlr/grammars-v4. GitHub.
<https://github.com/antlr/grammars-v4/blob/master/java/java9/Java9Parser.g4>